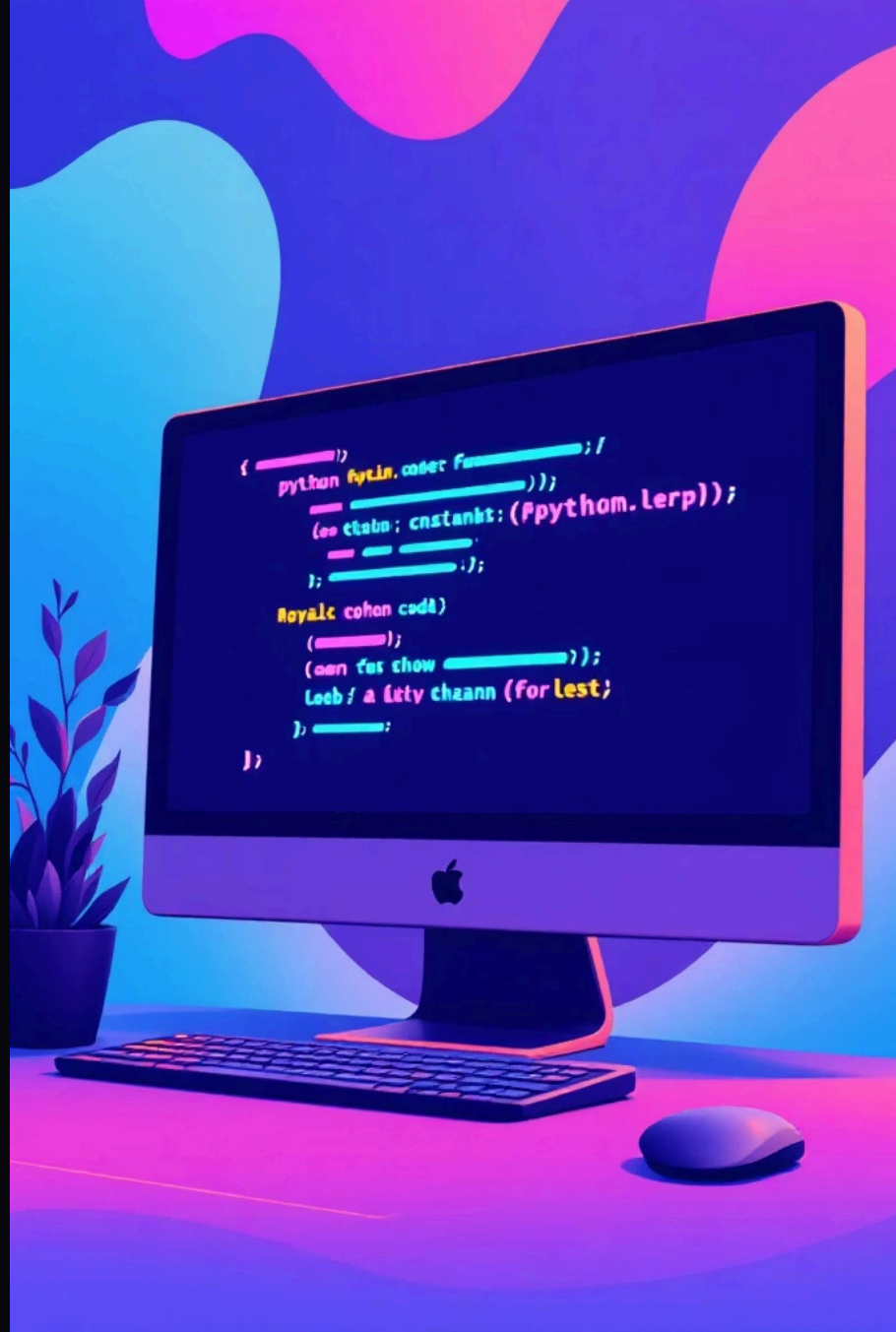
Loops and Lists in Python

Master the fundamentals of iteration and data collection in Python programming.



Introduction to Loops

What Are Loops?

Loops are control structures that let you repeat a block of code multiple times without writing it over and over.

Two Main Types

Python offers two primary loop types: **for loops** (when you know how many times to iterate) and **while loops** (when you loop based on a condition).

The For Loop

Basic Syntax

The for loop iterates through a sequence of numbers using the range() function.

```
for i in range(5):
    print(i)
```

Output

```
0
1
2
3
4
```

How It Works

The variable `i` takes values from 0 to 4. Each iteration, Python executes the indented code block once.

Task: Write a for loop that prints numbers 1 to 10.

The Range Function

1

Basic Range

`range(5)` generates 0, 1, 2, 3, 4

2

Start and Stop

`range(1, 11)` generates 1 through 10

3

Step Parameter

`range(2, 21, 2)` generates even numbers 2, 4, 6...

```
for i in range(1, 11):
    print(i)

# Prints: 1, 2, 3, 4, 5, 6, 7, 8, 9, 10
```

Task: Print even numbers from 1 to 20 using range().

The While Loop

Conditional Repetition

While loops execute code as long as a condition remains true. Use these when you don't know exactly how many iterations you need.

```
count = 1
while count <= 5:
    print(count)
    count += 1

# Output: 1, 2, 3, 4, 5
```

The loop continues until `count` exceeds 5.

Visual Flow

The condition is checked before each iteration. If true, the code runs; if false, the loop ends.

Task: Print the multiplication table of 5 (5, 10, 15, 20...50).

Infinite Loops and Break

Infinite Loop

A loop that never stops because its condition always evaluates to True.

Break Statement

Use `break` to exit a loop immediately, even if the condition is still true.

```
while True:
    print("Hello")
    break

# Prints "Hello" only once
```

Best Practice: Always include a break condition or modify the loop variable to avoid getting stuck in an infinite loop.

Introduction to Lists

A list is a collection that stores multiple values in a single variable. Lists are ordered, changeable, and allow duplicate values.

Ordered

Items have a defined position (index)

Changeable

Add, remove, or modify items

Allows Duplicates

Same value can appear multiple times

```
names = ["Ram", "Sam", "John"]
print(names)
# Output: ['Ram', 'Sam', 'John']
```

Indexing and Accessing Elements

Zero-Based Indexing

Python uses zero-based indexing, meaning the first element is at index 0.

```
names = ["Ram", "Sam", "John"]

print(names[0])  # Ram
print(names[1])  # Sam
print(names[2])  # John
```

Negative Indexing

Use negative numbers to access elements from the end of the list.

```
print(names[-1])  # John
print(names[-2])  # Sam
```

Task: Write code to print the last element of a list without knowing its length.

List Functions and Operations

Adding Items

`append()` adds to the end

```
names.append("Arun")
# names is now ["Ram",
"Sam", "John", "Arun"]
```

Removing Items

`remove()` deletes by value

```
names.remove("Sam")
# names is now ["Ram",
"John", "Arun"]
```

Getting Length

`len()` returns count

```
print(len(names))
# Output: 3
```

Looping Through Lists

Combine loops and lists to process all elements efficiently.

```
names = ["Ram", "Sam", "John", "Arun"]

for name in names:
    print(name)

# Output: Ram, Sam, John, Arun
```

01

Get User Input

Ask user to enter 3 names

02

Store in List

Add each name to a list

03

Print All Names

Loop through and display each name

Practice Activity: Create a program that collects 3 names from the user and prints them all using a for loop.